

Words of Wonders — Project Summary

A self-contained Java Swing word-puzzle desktop app

Contents

1. What This Project Is	1
2. Architecture: MVC	2
3. Login & Registration	3
4. The Dashboard	4
5. Gameplay: How Words of Wonders Works	4
6. Scoring, Hints & Finishing	5
7. Settings, Leaderboard & Encyclopedia	5
8. Admin Panel	6
9. Internationalization (English / Khmer)	6
10. Sound Design	6
11. Visual Design System (UITheme.java, 4,335 lines)	6
12. Data Files (data/*.json)	7
13. Testing	7
14. Project Layout	7
15. Known Limitations (documented honestly in the README)	8
16. Suggested Demo Flow (from the README's demo script)	8

1. What This Project Is

Words of Wonders is a desktop word-finding puzzle game built entirely in **Java Swing**. The player is given a small circle of 5–6 letters and must trace out every hidden word that can be spelled from them.

Key facts:

- **24 Java files, ~11,100 lines of code**
- **Zero external libraries** — no Maven, no Gradle, no internet required to run
- Compiles with a single `javac` command

- Everything is hand-written on top of the plain Java Standard Library: its own JSON reader/writer, its own generated sound effects (sine-wave synthesis, no audio files), and its own hand-drawn icon set (no image assets except one background photo)
- The only external touchpoint is Java's built-in `java.net.http` client, used for a handful of *optional* online features (a “did you know” fact, Wikipedia word lookups, a global leaderboard) — all of which fail silently and gracefully if there's no internet connection

Run it:

```
./run.sh          # macOS / Linux – build + launch
./run.sh test     # run the automated test suite (321 assertions)
run.bat          # Windows – build + launch (no test support)
```

The `-encoding UTF-8` flag is essential, since the app contains Khmer text — without it, the compiler mangles those characters.

Three demo accounts are auto-created on first run:

Username	Password	Role
admin	admin123	Administrator
bob	password456	Player
rithphea	password123	Player

A “Continue as Guest” option skips login entirely (progress not saved).

2. Architecture: MVC

The project follows the classic **Model–View–Controller** pattern:

Layer	Role	Analogy
Model	Holds data and business rules	The filing cabinet
View	Everything the user sees/clicks	The shop front
Controller	Receives user actions, tells the Model what to do	The manager

Golden rule: the View never touches a file directly — it always goes through the Controller, which goes through the Model. Example flow when a player finds the word *HEART*: the game screen (View) detects completion → updates the on-screen score → `GameController` reads the new total → `UserRepository` (Model) rewrites `data/users.json`.

Model layer

File	Lines	Purpose
<code>User.java</code>	142	One player: username, email, hashed password, points, admin flag
<code>UserRepository.java</code>	249	Loads/saves <code>users.json</code> ; hashes & checks passwords
<code>World.java</code>	46	A theme world (e.g. Ancient Egypt)
<code>Level.java</code>	68	A level within a world
<code>WowLevel.java</code>	81	A puzzle definition (JSON-backed)
<code>GameRepository.java</code>	162	Loads the three level data files

File	Lines	Purpose
------	-------	---------

Controller layer

File	Lines	Purpose
AuthController.java	125	Login, registration, validation, error messages
GameController.java	94	Points, leaderboard, admin user management

View layer

File	Lines	Purpose
MainUI.java	93	Main window; switches between screens
WelcomeScreen.java	537	Login / registration forms
Dashboard.java	457	Home screen after login
WordsOfWondersGameScreen.java	1,037	The game itself — largest feature file
SettingsModal.java	312	Theme / language / sound settings
LeaderboardModal.java	322	Top-players list
EncyclopediaModal.java	200	Wonder articles
AdminControlModal.java	439	Admin user management
UITheme.java	4,335	The entire design system (colours, fonts, spacing, components, icons)
UIUtil.java	142	Layout helpers
SoundUtil.java	257	Generates & plays sound effects

Util layer

File	Lines	Purpose
JsonUtil.java	386	Hand-written JSON reader/writer
ApiService.java	193	Optional online lookups
I18n.java	292	All 142 English/Khmer text strings

3. Login & Registration

The Welcome Screen has a Login tab, a Register tab, and a “Continue as Guest” button, plus a language toggle.

Registration validates seven things in order, stopping at the first failure: username not empty → email not empty → password not empty → username ≥ 3 chars → username uses only letters/numbers/_/.- → email matches a standard regex → password ≥ 4 chars. “Forgot password?” doesn’t email anyone (no mail server) — it just tells the user to ask their teacher.

Login checks: fields not empty → username exists (case-insensitive) → password matches.

Password security: passwords are stored as **SHA-256 hashes**, never in plain text (e.g. admin123 is stored as 240be518...4720a9). Login hashes the typed password and compares hashes. Legacy plain-text passwords are auto-upgraded to a hash the first time they’re used.

All accounts live in a single file, `data/users.json`. Saves are done safely — write to a temp file, then swap it into place, so a crash mid-write can't corrupt the file.

4. The Dashboard

Shown after login: an avatar, name and rank, a language toggle, quick-access icons (Encyclopedia, Settings, Hall of Fame, Admin Panel for admins, Logout), a points counter, and one large **Words of Wonders** hero card that opens the game. The card resizes responsively so nothing is ever clipped.

Rank thresholds (based on lifetime points):

Points	Rank
600+	Legendary
300+	Gold
150+	Silver
50+	Bronze
< 50	Novice

A “Did you know?” fact is optionally fetched online; if there's no connection, the line is simply left blank.

5. Gameplay: How Words of Wonders Works

You're given a circle of 5–6 letters and must find **9 hidden words** that can be spelled from them (letters may repeat only if they appear twice in the circle).

Example (Easy): letters E A R T H → hidden words: EARTH, HEART, HATER, TEAR, HEAR, RATE, HARE, TARE, HART.

The eight puzzles

#	Word	Letters	Difficulty tile
1	EARTH	5	Easy
2	GARDEN	6	Medium
3	LISTEN	6	—
4	CASTLE	6	—
5	PLANET	6	—
6	GUITAR	6	Hard
7	WONDER	6	—
8	FROZEN	6	—

Only **3 of the 8** (Easy/Medium/Hard) are reachable from the difficulty picker; the rest only appear via “Play Again,” which picks a random puzzle from all eight.

Input methods: drag across letters in the circle, or tap them one at a time — both work identically. Mistakes can be undone by dragging backward or tapping an earlier letter; “Clear” wipes the selection. Shuffling the letter positions is free and unlimited.

The key mechanic — chained words: if the word you just found is the *start* of another unfound word (e.g. HEAR → HEART), your selection stays alive so you can keep extending it instead of starting over. This is the design’s signature feel.

Bonus words: any other valid word from the same letters also scores points (+10), tracked in a separate “Bonus Jar.” They don’t count toward puzzle completion. A built-in 229-word dictionary is checked first; unknown words are verified live against Wikipedia and then remembered for the session.

The word-progress grid on the left shows each target word as boxes with three states: unfound (dark/hidden), hinted (one amber letter), and found (full teal row with the word revealed).

6. Scoring, Hints & Finishing

Action	Points
Find a hidden word	+20
Find a bonus word	+10
Use a Hint	−15
Use “Reveal Word”	net −10 (button shows “−30”, but a completed word also grants the usual +20)
Shuffle / Clear	0

Max score for one puzzle: 180 (9×20) plus 10 per bonus word found. Scores *can* go negative through heavy power-up use.

Both power-ups are locked until you’ve found at least one word yourself (a fresh puzzle starts at 0, so you can’t go negative immediately).

Finishing: once all 9 words are found (by hand or via power-ups), the board locks, a victory fanfare plays, confetti animates (170 particles), points are saved, and a completion screen appears with your **title**, based only on hint letters used:

Hint letters used	Title
0	PERFECT (3/3)
1–2	GREAT JOB (2/3)
3+	SOLVED (1/3)

(Note: using “Reveal Word” doesn’t count against this rating — a player who reveals every word still gets PERFECT. This is a known/documented gap.)

From the completion screen: **Play Again** (random puzzle), **Change Difficulty**, or **Back to Dashboard**.

7. Settings, Leaderboard & Encyclopedia

Settings: four options — Theme (4 palettes: Midnight Nebula (default), Deep Ocean, Royal Amethyst, Daylight Crystal — the only light theme), Language (English/Khmer), Sound on/off, and a Test Sound button. Theme switches repaint instantly since every colour is read live from the active palette.

Leaderboard (“Hall of Fame”): a *Local* tab (top 10 from the local users.json, admins excluded, ties broken alphabetically) and a *Global* tab (online shared scoreboard, with a clearly-labelled offline fallback if there’s no connection).

Encyclopedia: six articles, one per world theme (Great Pyramid of Giza, The Solar System, Coral reefs, Tyrannosaurus, Castles, The Amazon rainforest), each fully written in both English and Khmer so it works offline; a live Wikipedia summary/thumbnail is layered on top when online.

8. Admin Panel

Visible only to admin accounts (isAdmin flag, or username admin specifically). Shows a table of all users (ID, Username, Email, Points, Rank, Role) with five actions: **Create User**, **Edit User**, **Reset Points**, **Add Points**, **Delete User** — plus Refresh/Close. Duplicate usernames/emails and blank usernames are rejected.

9. Internationalization (English / Khmer)

All 142 UI strings live in one place (I18n.java), registered as English/Khmer pairs and looked up by key (e.g. I18n.get("btn_login")). No screen hard-codes text, so the whole app switches language instantly via a globe-icon toggle (present on Welcome Screen, Dashboard, and Settings). Placeholders like {0} are filled with live values (e.g. “You found 3 / 9 words”). Missing translations fall back to English; a missing key entirely just shows the key name (useful for catching bugs).

Note: language, theme, and sound settings are *not* persisted — they reset to defaults every time the app restarts.

10. Sound Design

No audio files exist in the project — every sound is synthesized as a waveform (44.1kHz, 16-bit, mono) at the moment it plays, generated on background threads so the UI never stutters.

Sound	Trigger	Character
Click	Any button press	Short 750 Hz blip
Letter	Selecting each letter	Bell pitch rises per letter (C, D, E, G, A, C, D, E) — long words “sing”
Correct	Finding a hidden word	Four bright notes (C, E, G, C)
Error	Mistake / insufficient points	Two descending sweeps
Hint	Power-up used / bonus word found	Rising sparkle
Shuffle	Re-arranging letters	A whoosh
Victory	Puzzle completed	Five-note fanfare

Missing audio hardware is caught silently — the game never crashes over sound.

11. Visual Design System (UITheme.java, 4,335 lines)

This single file is the project’s entire visual language: colour palette, spacing scale, font sizes, and a library of reusable components (primaryButton, card, glowCard, badge, progressBar, letterTilesRow, avatar, etc.), so every screen looks consistent.

- **22 hand-drawn icons** (VectorIcon), built purely from Java 2D shape calls — no image files, always sharp at any size.

- **Four custom-painted components:** the letter circle (glow lines, drag trail, radial background), the word grid, the animated progress bar, and a drifting animated background (different colour sets per screen).
- **Responsive scaling:** the whole UI scales proportionally between 0.85×–1.5× as the window resizes, plus per-component min/max sizing so nothing overflows on small screens. The gradient title auto-shrinks its font if it doesn't fit (checked in both languages).

12. Data Files (data/*.json)

File	Contents
users.json	Live player accounts (the only one actually read/written during normal play)
worlds.json	6 theme worlds (Ancient Egypt, Outer Space, Deep Ocean, Dinosaur World, Medieval Kingdoms, Rainforest Adventure)
levels.json	18 levels (3 per world), each with difficulty and point reward
wow_levels.json	18 puzzle <i>definitions</i> (word lists per theme)

Important honesty note from the docs: the 8 puzzles you actually play (EARTH, GARDEN, LISTEN, CASTLE, PLANET, GUITAR, WONDER, FROZEN) are hard-coded directly in the game screen file — they are **not** loaded from wow_levels.json, and editing that file changes nothing in-game. The three level files are validated by the test suite and are a working demonstration of the data layer, but aren't wired into actual gameplay.

13. Testing

One file, SmokeTest.java (1,104 lines), runs **321 assertions**, exits non-zero on any failure, and prints ALL CHECKS PASSED (321 assertions) on success. Coverage includes:

1. User CRUD, points, leaderboard ordering, admin exclusion
2. All JSON data files load/parse correctly
3. Every text key has both English and Khmer entries; no orphaned lookups
4. Responsive layout checks (hero card, Settings dialog) across six window widths, in both languages
5. **Pixel-level rendering checks** of every screen across all four themes
6. Repeated language switching doesn't leak duplicate UI components
7. **Full automated playthroughs of all 8 puzzles**, alternating drag/tap input, via real MouseEvents through the real hit-testing code
8. Hint/Reveal Word power-up gating and exact point math

14. Project Layout

```

Json_Game/
├─ run.sh / run.bat
├─ src/com/worldofwonder/
│   ├─ Main.java
│   ├─ controller/    (AuthController, GameController)
│   └─ model/         (User, UserRepository, World, Level, WowLevel, GameRepository)

```

```

|   |─ util/          (JsonUtil, ApiService, I18n)
|   |─ view/         (MainUI, WelcomeScreen, Dashboard, WordsOfWondersGameScreen,
|   |                 SettingsModal, LeaderboardModal, EncyclopediaModal,
|   |                 AdminControlModal, UITheme, UIUtil, SoundUtil)
|   |─ test/         (SmokeTest)
|─ data/             (users.json, worlds.json, levels.json, wow_levels.json, cache/avatars/)
|─ assets/images/Game4.jpeg

```

Largest files: UITheme.java (4,335 lines) > SmokeTest.java (1,104) > WordsOfWondersGameScreen.java (1,037) > WelcomeScreen.java (537) > Dashboard.java (457) > AdminControlModal.java (439).

15. Known Limitations (documented honestly in the README)

Gameplay - “Difficulty” is really just a puzzle selector — no timer, lives, or multiplier; Medium and Hard both give 6 letters / 9 words. - Only 3 of 8 puzzles are reachable by difficulty choice; the rest need “Play Again” (random). - A bonus word’s points get saved **twice** (once immediately, once again in the final total that already includes it) — effectively worth 20, not 10, in the saved total. - The completion rating ignores the “Reveal Word” power-up entirely. - “Reveal Word” is labelled “-30” but nets -10; its icon (magnifier) doesn’t match its “hammer”-like function. - Leaving mid-puzzle via Back gives no confirmation and doesn’t save, though the board state persists if you return. - “New Puzzle” actually just reopens the difficulty picker.

Data - levels.json/worlds.json/wow_levels.json are unused by actual gameplay (see §12). - The words field in wow_levels.json is a JSON string containing a stringified array — never actually decoded. - **Known bug:** two separate in-memory copies of the user list exist (one for login/registration, one for saving points/admin actions), which can cause one side to overwrite the other’s changes — fine for single-user demos, risky for real multi-user use. - A malformed users.json (e.g. a stray comma) crashes the app on startup — only file I/O errors are caught, not parse errors. - Launching from the wrong working directory silently creates a fresh empty data/ folder (mitigated by the launch scripts, which cd first). - The admin account has two hard-coded passwords (admin123, admin67) that always work regardless of the stored hash — intentional for demo reliability, but means the admin password can’t truly be changed.

Interface - Switching language doesn’t retroactively update difficulty tiles or in-game buttons until you leave and re-enter. - Theme/language/sound settings reset on every app restart. - Global leaderboard, “did you know” facts, Wikipedia word checks, and encyclopedia summaries all require internet and fail quietly offline. - One tooltip is untranslated; two of the 142 text keys (btn_later, wow_solved_sub) are defined but never actually displayed. - Some dead code remains from removed features (an unused hearts/hintCoins system, unused GameController methods, a leftover “spin the letters” subtitle from a removed spin-wheel feature).

What’s verified solid: all 321 automated assertions pass, the build is warning-free, all 8 puzzles are completable end-to-end by both input methods, and the test suite actively fails if removed features (wheel, daily rewards) reappear.

16. Suggested Demo Flow (from the README’s demo script)

1. **Login screen** (30s) — register, log out/in, show the hashed password in users.json.
2. **Dashboard** (30s) — show rank/points, switch themes and language live.
3. **Start a game** (15s) — Easy puzzle; note the tile’s letter/word counts come from real data.
4. **Core mechanic** (90s) — find HEAR, then extend it to HEART without resetting; find a bonus word.
5. **Power-ups** (30s) — show Hint locked at 0 points, then use it after finding a word.
6. **Shuffle & undo** (30s) — free shuffle; drag-back undo.
7. **Word grid** (20s) — point out the three visual states and the progress bar.
8. **Finish** (30s) — confetti, fanfare, saved score, and the (honestly limited) rating.
9. **Other screens** (45s) — Leaderboard, Encyclopedia (offline Khmer + live Wikipedia), Admin Panel as admin.

10. **Wrap-up** (30s) — emphasize: pure Java Swing/MVC, zero external libraries, generated sound/icons, full bilingual support, 321 passing automated checks including full playthroughs and pixel-level theme checks.

Summary generated from the project's own README documentation.